

Ranking de Algoritmos Optimizados para *push_swap* · Caso de 100 números

Comparativa de las estrategias usadas por estudiantes de la Escuela 42 — recopilada de GitHub, YouTube, Substack/Medium, Discord/Slack y LinkedIn. Objetivo de evaluación 42: < **700 movimientos para 100** y < **5 500 para 500**.

Nota metodológica: los números de movimientos para 100 son medias aproximadas reportadas por los autores y comunidad (varían según el set aleatorio). La columna **Potencial de optimización** distingue lo que mejora el algoritmo **AISLADO** (por sí mismo) frente a lo que aporta **COMBINADO** con otras técnicas.

- TIER S** Óptimo para puntuación 125/125 (≤ ~550 mov.)
- TIER A** Aprueba con holgura (≤ ~700 mov.)
- TIER B** Aprueba justo / mejorable (~700-1100)
- TIER C** Funcional pero no apto para nota máxima

#	ALGORITMO / FAMILIA	OBJETIVO (MOV. ~100)	PROTOTIPO (PSEUDOCÓDIGO)	POTENCIAL DE OPTIMIZACIÓN · AISLADO VS COMBINADO	FUENTE (ESTUDIANTE / PLATAFORMA)
1	Three-Way Quick Sort sobre 4 posiciones Familia: Divide & conquer / Quicksort recursivo TIER S	~~ 500-620 (100) 3 784 mov. medio (500) de los más eficientes con código público	<pre>map valores → rango (1..N) recursive_sort(chunk): if base_case: static_sort(); return split(chunk) → max→TOP_A mid→TOP_B min→BOTTOM_B recursive_sort(max) recursive_sort(mid) recursive_sort(min)</pre>	AISLADO Usa las 4 "posiciones" (TOP/BOTTOM de A y B); extender el <i>base case</i> a chunks de 2-3-4; pivotes dinámicos según tamaño; "Sort ASAP" y "Easy sort" (saltar tramos ya ordenados). COMBINADO + post-procesado de operaciones (fusionar <i>sa+sb→ss</i> , <i>ra+rb→rr</i>) y purga de operaciones neutras (<i>ra+rra</i>). Pasa de 5 636 → 3 784 en 500.	Ulysse Gerkens (Medium/Substack + GitHub ulsgks) Referencia 42-Seoul jeongble (Slack/Discord, 3 720/500)
2	Greedy "Cost Function" (Turk Algorithm) Familia: Selección por coste mínimo TIER A	~~ 600-680 (100) Fiable para 125/125	<pre>pb x2 (semilla en B) while size_a > 3: for each n in A: cost = rot_a + rot_b (usar rr/rrr si mismo signo) track cheapest ejecutar movimientos cheapest; pb sort_3(A) while B: insertar en pos correcta; pa rotar A hasta min en top</pre>	AISLADO Calcular coste combinando rotaciones simultáneas (<i>rr/rrr</i>) cuando A y B rotan en el mismo sentido; elegir SIEMPRE el número más barato antes de cada push. COMBINADO + sort de 3 hardcodeado + colocación "encima del máx/mín" en B + post-optimización de operaciones. Muy compatible con la idea de chunks.	A. Yigit Ogun (Medium, 125/125) + GitHub byaliego alx-sch "Enhancing Turk" (GitHub)
3	Chunk / Insertion Sort por bloques Familia: Particionado por rangos TIER A	~~ 650-700 (100) ~5 chunks para 100	<pre>dividir rango en k chunks (≈5 para 100, ≈11 para 500) for cada chunk (de menor a mayor): buscar nº del chunk en A (desde top y desde bottom) elegir el de menos rot (ra/rra) pb (insertando ordenado en B) while B: traer max al top; pa</pre>	AISLADO Afinar el nº de chunks empíricamente (5 para 100, 11 para 500); decidir ra vs rra según si el índice < mitad de la pila; insertar manteniendo B parcialmente ordenado. COMBINADO + cost-function al devolver de B→A + sort_3/sort_5 hardcodeado para los restos. Base habitual del 90-125/100.	Jamie Dawson (Medium/Substack, 42 Silicon Valley) AlessandroMetta (GitHub, 90/100)
4	L.I.S. (Longest Increasing Subsequence) Familia: Programación	~~ 680-750 (100) 125/100 reportado	<pre>lis = longest_increasing_subseq(A) mantener LIS en A; pb del resto a B while B: para cada n en B: calcular mov_b</pre>	AISLADO Mantener la subsecuencia más larga ya ordenada en A reduce drásticamente los push; tabla de combinación de signos <i>mov_a/mov_b</i> para usar <i>rr/rrr</i> . COMBINADO + cost-function en la reinserción B→A +	Mattia Cerchi (sisittu99) README L.I.S. (GitHub, 125/100) Slack 42 (@mcerchi)

	dinámica + greedy TIER A	<pre>para cada n en B: calcular mov_a y mov_a (nº inmediato mayor en A) combinar signos (tabla rr/rrr) elegir mejor; pa alinear A (min al top)</pre>	COMBINADO + cost function en la remoción de A y circular buffer (array) en vez de listas enlazadas para velocidad. Riesgo de TLE si no se optimiza.		
5	Radix Sort binario (bitwise) Familia: No comparativo / por dígitos TIER B	~~700-1100 (100) Simple pero menos óptimo	<pre>normalizar valores → índices 0..N-1 max_bits = bits(N-1) for bit in 0..max_bits: for i in 0..size_a: if (top & (1<<bit)) == 0: pb else: ra while B not empty: pa // queda ordenado al final</pre>	AISLADO Mapear valores a su índice reduce los bits a procesar (complejidad $\propto$ n° de bits); usar rr/rrr donde aplique. Muy fácil de implementar y depurar. COMBINADO Poca sinergia: procesa todos los bits aunque ya esté ordenado. Conviene combinar con sort_3/sort_5 hardcodeado para n pequeños, donde radix es ineficiente.	arommers (GitHub, Codam/42) · hu8813 devnyxie "Bitwise Radix Sort" (notes/Substack)
6	Selection / Simple Insertion Sort Familia: Comparativo ingenuo (baseline) TIER C	~> 2 000 (100) No aprueba 100/500	<pre>while A no vacío: llevar el menor de A al top (ra/rra) pb while B no vacío: pa // solo válido como punto de partida</pre>	AISLADO Elegir ra vs rra según posición del mínimo reduce algo el coste, pero el techo es muy alto. Solo sirve como referencia conceptual. COMBINADO Su valor real es evolucionarlo: añadir chunks lo convierte en el algoritmo #3; añadir cost-function lo convierte en el #2.	Jamie Dawson (versión inicial, Medium) Discusiones Reddit r/42_school / StackOverflow

Fuentes consultadas (estudiantes 42 y comunidad): Ulysse Gerkens — [Medium «Push Swap in less than 4200 operations»](#) & GitHub ulsgks · A. Yigit Ogun — [Medium «A journey to find most efficient sorting algorithm»](#) & GitHub byaliego · Jamie Dawson — [Medium «The least amount of moves with two stacks»](#) · Mattia Cerchi — [GitHub README L.I.S.](#) · aroppers — [GitHub Radix Sort](#) · Anass Outiskte — [LinkedIn «The Art of Sorting with Two Stacks»](#) · YouTube (Fred «100 & 500 high score», Aaron R «Demo») · 42 Slack/Discord (jeongble 3 720/500).

Documento generado como guía comparativa. Los movimientos son aproximados y dependen del conjunto aleatorio de entrada y de las micro-optimizaciones implementadas.